

MANUAL TÉCNICO Y DIDÁCTICO

Configuración de un Agente de Inteligencia Artificial Defensivo y Académico en Kali Linux

Curso de Seguridad Informática

Laboratorio universitario · Entorno controlado y legal

Dato	Valor de referencia
Fecha de elaboración	15 de junio de 2026
Sistema operativo	Kali Linux 2026.1 (rolling) · kernel 6.18
Python	3.12.x (confirmar con: python3 --version)
Gestor de paquetes Python	pip 24.x + venv
Tecnología del agente	Python 3 puro (sin framework externo)
Modelo (API, opción A)	Proveedor remoto vía API (ej. claude-haiku-4-5)
Modelo (local, opción B)	Ollama + modelo pequeño (ej. llama3.2:3b)
Dependencias clave	requests, python-dotenv
Clasificación de uso	Educativo / defensivo / no ofensivo

Las versiones son de referencia. Confirme cada una en la documentación oficial antes de ejecutar los procedimientos, ya que Kali es de tipo «rolling release» y puede cambiar.

Tabla de contenido

Introducción	5
1. Objetivos del manual	5
1.1 Objetivo general	5
1.2 Capacidades del agente	5
1.3 Restricciones explícitas	5
2. Perfil de los usuarios	5
3. Entorno de implementación	6
3.1 Verificación del entorno	6
4. Arquitectura del agente	6
4.1 Diagrama de comunicación entre componentes	7
5. Alternativas de modelo de IA	7
5.1 Opción A: modelo mediante API	7
5.2 Opción B: modelo local con Ollama	8
6. Tecnologías y dependencias	8
6.1 Por qué Python puro	9
6.2 Dependencias	9
7. Estructura del proyecto	9
8. Procedimiento paso a paso	11
Paso 1: Crear la máquina virtual	11
Paso 2: Crear el snapshot	11
Paso 3: Actualizar Kali Linux	11
Paso 4: Instalar paquetes base	12
Paso 5: Crear el directorio de laboratorio	12
Paso 6: Crear el entorno virtual	13
Paso 7: Instalar dependencias	13
Paso 8: Configurar el modelo	13
Paso 9: Crear el agente	14
Paso 10: Crear herramientas seguras	14
Paso 11: Implementar la confirmación humana	15
Paso 12: Implementar la lista blanca	15
Paso 13: Configurar los logs	15
Paso 14: Ejecución inicial y prueba de conexión	15
Paso 15: Pruebas	16
Paso 16: Cierre y restauración del entorno	16
9. Código fuente	17

9.1 Lista blanca y análisis de riesgo.....	17
10. Prompt del sistema del agente.....	18
11. Controles de seguridad.....	19
11.1 Por qué un agente con acceso a terminal es un riesgo	19
12. Casos de prueba	20
13. Práctica académica.....	21
13.1 Título.....	21
13.2 Objetivo general.....	21
13.3 Objetivos específicos	21
13.4 Requisitos previos.....	21
13.5 Recursos.....	21
13.6 Normas de seguridad.....	21
13.7 Escenario.....	21
13.8 Actividades paso a paso (2 a 3 horas).....	21
13.9 Preguntas de análisis	22
13.10 Evidencias solicitadas	22
13.11 Entregables.....	22
13.12 Conclusiones (a redactar por el estudiante)	22
13.13 Criterios de evaluación y rúbrica (100 puntos).....	22
14. Solución de problemas.....	23
15. Desinstalación y reversión	23
Conclusiones.....	25
Glosario.....	25
Referencias.....	25
Anexo A. Código fuente completo	26
config.py	26
security.py	28
tools.py.....	30
agent.py.....	32
app.py.....	35
tests/test_security.py	37
Anexo B. requirements.txt	40
Anexo C. .env.example.....	40
Anexo D. .gitignore.....	40
Anexo E. Lista de comprobación para el profesor.....	41
Anexo F. Lista de comprobación para el estudiante	41

Anexo G. Rúbrica de evaluación (100 puntos)	41
---	----

Introducción

Este manual guía a estudiantes universitarios de Seguridad Informática en la construcción, desde cero, de un agente de inteligencia artificial funcional sobre Kali Linux. El agente recibe instrucciones en lenguaje natural, razona sobre ellas con un modelo de lenguaje y puede ejecutar —solo tras confirmación humana— un conjunto reducido de comandos informativos y defensivos previamente autorizados.

El enfoque es deliberadamente conservador. Un agente de IA con acceso a una terminal es una herramienta poderosa, pero también un riesgo de seguridad: si se le otorga demasiada libertad, podría ejecutar acciones no previstas. Por eso el diseño prioriza el principio de mínimo privilegio, una lista blanca estricta de comandos, la confirmación humana obligatoria y un registro de auditoría completo.

⚠ Uso responsable: Todo el laboratorio se ejecuta únicamente dentro de una máquina virtual aislada. El agente no ataca sistemas externos, no explota vulnerabilidades, no elimina archivos, no escala privilegios ni recopila credenciales. Su finalidad es exclusivamente educativa y defensiva.

1. Objetivos del manual

1.1 Objetivo general

Permitir que un estudiante configure, desde cero, un agente de IA seguro en Kali Linux y comprenda tanto los procedimientos técnicos como los conceptos de seguridad que los sustentan.

1.2 Capacidades del agente

Al finalizar, el agente será capaz de:

- Recibir instrucciones escritas en lenguaje natural.
- Analizar información del sistema operativo mediante comandos informativos.
- Ejecutar únicamente comandos previamente autorizados (lista blanca).
- Consultar documentación técnica y explicar comandos y resultados.
- Leer y analizar archivos dentro de un directorio de laboratorio (workspace).
- Mantener un historial básico de las interacciones.
- Rechazar instrucciones peligrosas o fuera del alcance autorizado.
- Operar exclusivamente dentro de un entorno académico, controlado y legal.

1.3 Restricciones explícitas

El agente NO debe, en ninguna circunstancia: ejecutar ataques automáticos, explotar vulnerabilidades, eliminar archivos, escalar privilegios, establecer persistencia, evadir controles de seguridad, extraer credenciales ni actuar contra sistemas externos.

2. Perfil de los usuarios

El manual está dirigido a estudiantes universitarios con conocimientos básicos o intermedios en Linux, Kali Linux, terminal y comandos, Python, redes y seguridad informática. Cada procedimiento se explica con suficiente detalle para que pueda seguirlo una persona que nunca haya configurado un agente de IA.

3. Entorno de implementación

Se utiliza el siguiente ambiente de referencia:

- Kali Linux actualizado (2026.1 al momento de redacción).
- Máquina virtual en VMware Workstation o VirtualBox.
- Usuario estándar con permisos sudo (no se trabaja como root).
- Python 3, pip y entornos virtuales (venv).
- Git, y Visual Studio Code o el editor Nano.
- Conexión a Internet (al menos para la instalación inicial).
- Mínimo 8 GB de RAM recomendados (más si se usa modelo local).
- Snapshot de la máquina virtual ANTES de comenzar.

3.1 Verificación del entorno

Ejecute estos comandos en la terminal de Kali para verificar versiones, memoria, disco y conectividad. Cada uno se explica a continuación.

```
cat /etc/os-release      # version de Kali Linux
uname -r                 # version del kernel
python3 --version        # version de Python 3
pip3 --version           # version de pip
git --version            # version de Git
free -h                  # memoria RAM disponible
df -h /                   # espacio libre en disco
ping -c 3 deb.debian.org # conectividad de red
```

Resultado esperado: cada comando devuelve una versión o un valor numérico; el «ping» muestra respuestas con tiempos en milisegundos. Si algún comando falla, consulte la sección de solución de problemas.

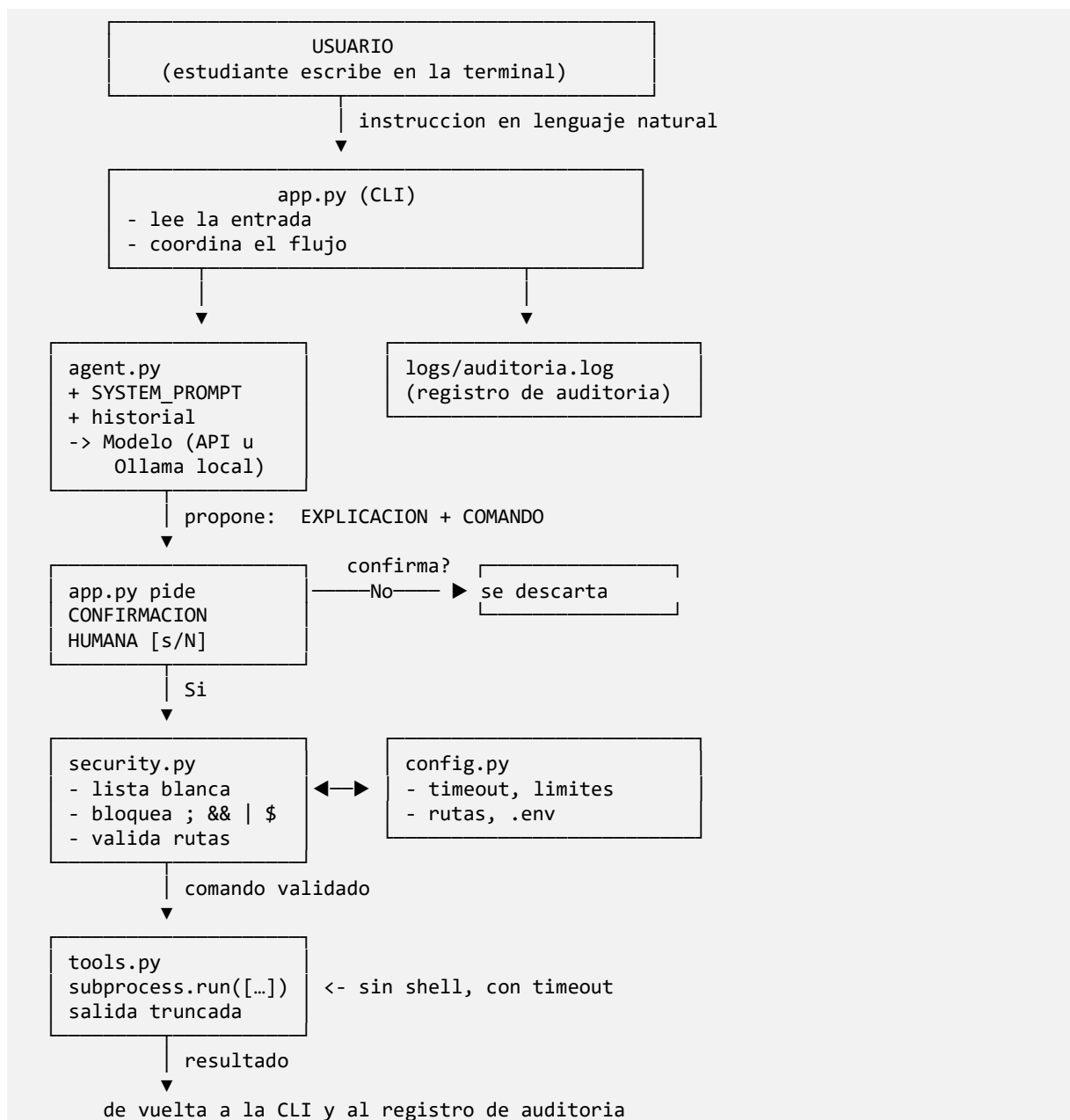
 **[CAPTURA]** Insertar captura de la salida de estos comandos de verificación.

4. Arquitectura del agente

La arquitectura es sencilla, segura y comprensible. Está compuesta por diez componentes:

1. Interfaz de línea de comandos (CLI): recibe la instrucción del usuario.
2. Modelo de lenguaje: interpreta la instrucción (API remota u Ollama local).
3. Prompt del sistema: define el rol, el alcance y las prohibiciones del agente.
4. Módulo de herramientas (tools): ejecuta acciones seguras.
5. Lista blanca de comandos: único conjunto de comandos permitidos.
6. Registro de actividades (logs): auditoría de cada interacción.
7. Directorio de trabajo restringido (workspace): único lugar de lectura de archivos.
8. Confirmación humana: ninguna ejecución ocurre sin un «sí» explícito.
9. Manejo seguro de claves API: variables de entorno, nunca en el código.
10. Mecanismo para detener la ejecución: timeout y cancelación manual.

4.1 Diagrama de comunicación entre componentes



Idea clave: El modelo solo SUGIERE; la persona AUTORIZA; el código VALIDA y EJECUTA. Esta separación de responsabilidades es el corazón de la seguridad del agente.

5. Alternativas de modelo de IA

5.1 Opción A: modelo mediante API

Un proveedor remoto expone el modelo a través de una API HTTP. El agente envía la conversación y recibe la respuesta. La clave de acceso se guarda en una variable de entorno dentro de un archivo .env que nunca se versiona.

Pasos (resumen; el detalle está en la Sección 8)

- Crear y activar el entorno virtual de Python.
- Instalar dependencias (requests, python-dotenv).
- Configurar el archivo .env con la clave y el modelo.
- Crear .gitignore para excluir .env y logs.
- Probar la conexión con una solicitud mínima.
- Manejar errores de autenticación (401/403) y límites (429).
- Controlar costos: limitar max_tokens y la longitud del historial.

⚠ Costos: Las APIs cobran por tokens. Para un laboratorio, limite max_tokens (p. ej. 1024), reinicie el historial entre ejercicios con /limpiar y evite enviar archivos largos al modelo.

5.2 Opción B: modelo local con Ollama

Ollama descarga y sirve modelos en la propia máquina. No hay costo por token ni se envían datos a terceros, pero el rendimiento depende del hardware.

Requisitos de hardware orientativos

Tamaño de modelo	RAM recomendada	Notas
1B–3B (p. ej. llama3.2:3b)	8 GB	Apto para laboratorio; respuestas rápidas en CPU.
7B–8B	16 GB	Mejor calidad; más lento sin GPU.
13B o más	32 GB+ / GPU	No recomendado para el laboratorio estándar.

Ventajas y limitaciones frente a la API

Aspecto	API remota (A)	Local con Ollama (B)
Costo	Por tokens	Sin costo por uso
Privacidad de datos	Salen de la máquina	Permanecen locales
Calidad de respuesta	Generalmente superior	Buena con límites
Requisitos de hardware	Bajos	Altos (RAM/CPU/GPU)
Conexión a Internet	Necesaria en uso	Solo para descargar el modelo
Complejidad de montaje	Baja	Media

Recomendación: Para un laboratorio universitario se recomienda la Opción B (modelo local con Ollama y un modelo pequeño como llama3.2:3b). Razones: no genera costos por token, mantiene los datos dentro de la VM, no requiere distribuir claves API entre estudiantes y funciona sin conexión durante la práctica. La Opción A queda como alternativa cuando el hardware es limitado o se busca mayor calidad de respuesta.

6. Tecnologías y dependencias

Para construir el agente se elige Python 3 puro, sin un framework de agentes (como LangChain, LangGraph o AutoGen). La justificación es pedagógica y de seguridad.

6.1 Por qué Python puro

Criterio	Python puro (elegido)	Framework de agentes
Transparencia	Cada control de seguridad es visible y auditable	La lógica de seguridad queda oculta en capas internas
Complejidad	Baja: pocos archivos, fácil de leer	Alta: abstracciones, muchas dependencias
Dependencias	Mínimas (requests, dotenv)	Numerosas y de cambio frecuente
Compatibilidad con Kali	Total	Variable según versiones
Objetivo didáctico	El estudiante entiende cada línea	Se pierde el control fino del comportamiento

Desventaja: hay que escribir manualmente la integración con el modelo y el bucle de conversación. En un agente de producción, un framework podría acelerar el desarrollo; aquí preferimos el control total, porque el foco es la seguridad y la comprensión.

6.2 Dependencias

```
requests==2.32.3      # cliente HTTP (API remota y Ollama)
python-dotenv==1.0.1  # carga de variables desde .env
```

⚠ Versiones: Las versiones fijadas son una referencia estable. Confirme en PyPI o en la documentación oficial antes de actualizar, ya que pueden cambiar.

7. Estructura del proyecto

El proyecto se organiza en módulos pequeños y enfocados. Esta separación facilita la lectura, las pruebas y la auditoría de seguridad.

```
agente-ia-kali/
├── app.py           # CLI principal: flujo y confirmacion humana
├── agent.py         # Logica del modelo y prompt del sistema
├── tools.py         # Herramientas seguras (ejecucion de comandos)
├── security.py      # Lista blanca, validacion, bloqueo de operadores
├── config.py        # Carga de configuracion y limites
├── requirements.txt # Dependencias de Python
├── .env             # Secretos reales (NO se versiona)
├── .env.example     # Plantilla de configuracion (si se versiona)
├── .gitignore       # Exclusiones de Git (.env, logs, venv)
├── logs/            # Registros de auditoria
├── workspace/       # Unico directorio de archivos accesible
├── tests/           # Pruebas unitarias de seguridad
└── README.md        # Guia rapida
```

Archivo / carpeta	Función
app.py	Punto de entrada. Lee la entrada, consulta al agente, pide confirmación y registra.
agent.py	Construye el prompt del sistema, llama al modelo (API u Ollama) y guarda el historial.
tools.py	Ejecuta comandos validados con subprocess.run (sin shell), aplica timeout y trunca salida.
security.py	Define la lista blanca, bloquea operadores peligrosos y valida que las rutas estén en workspace.
config.py	Centraliza configuración y límites; lee variables de entorno; valida antes de arrancar.
requirements.txt	Lista de dependencias con versiones fijadas.
.env / .env.example	Secretos reales / plantilla. El .env real nunca se sube al repositorio.

Archivo / carpeta	Función
.gitignore	Evita versionar .env, logs y el entorno virtual.
logs/	Registros de auditoría (fecha, usuario, solicitud, comando, resultado).
workspace/	Único directorio donde el agente puede leer archivos.
tests/	Pruebas que verifican los controles de seguridad.
README.md	Instrucciones rápidas de instalación y uso.

8. Procedimiento paso a paso

Cada paso indica objetivo, explicación, comandos, lugar de ejecución, resultado esperado, verificación y errores comunes con su solución.

Paso 1: Crear la máquina virtual

Objetivo: Disponer de un entorno aislado donde el laboratorio no afecte al equipo anfitrión.

Explicación conceptual: Una máquina virtual (VM) ejecuta un sistema operativo completo dentro de otro. El aislamiento es la primera capa de seguridad: cualquier error queda contenido en la VM.

Dónde se ejecuta: Hipervisor (VMware Workstation o VirtualBox) en el equipo anfitrión.

```
# Acciones en la interfaz grafica del hipervisor:
# 1. Nueva VM -> seleccionar ISO de Kali Linux 2026.1
# 2. Asignar 8 GB de RAM (16 GB si usara modelo local grande)
# 3. Asignar 2+ nucleos de CPU y 40 GB de disco
# 4. Configurar la red en modo NAT o red interna (NUNCA puente)
```

Resultado esperado: La VM arranca en el escritorio de Kali Linux.

Cómo verificar: Inicie sesión y abra una terminal.

Errores comunes y solución:

- La VM no arranca: verifique que la virtualización (VT-x/AMD-V) esté habilitada en la BIOS/UEFI.
- Rendimiento lento: aumente la RAM asignada o reduzca el tamaño del modelo local.

 **[CAPTURA]** Configuración de red de la VM mostrando modo NAT o red interna.

⚠ Red: Use SIEMPRE modo NAT o red interna. El modo puente expone la VM a la red física y está prohibido durante las prácticas.

Paso 2: Crear el snapshot

Objetivo: Guardar un punto de restauración limpio antes de instalar nada.

Explicación conceptual: Un snapshot congela el estado de la VM. Si algo sale mal, se restaura en segundos sin reinstalar.

Dónde se ejecuta: Menú del hipervisor (VM → Snapshot → Tomar snapshot).

```
# Interfaz grafica:
# VMware: VM > Snapshot > Take Snapshot... (nombre: 'base-limpia')
# VirtualBox: Máquina > Tomar instantanea... (nombre: 'base-limpia')
```

Resultado esperado: Aparece un snapshot llamado «base-limpia» en la lista.

Cómo verificar: Revise el administrador de snapshots del hipervisor.

Errores comunes y solución:

- No hay espacio en disco del anfitrión: libere espacio antes de tomar el snapshot.

Paso 3: Actualizar Kali Linux

Objetivo: Tener el sistema y los repositorios al día.

Explicación conceptual: Kali es «rolling release»: se actualiza de forma continua. «apt update» refresca la lista de paquetes y «full-upgrade» instala las versiones nuevas.

Dónde se ejecuta: Terminal de Kali.

```
sudo apt update
sudo apt full-upgrade -y
```

Resultado esperado: apt descarga e instala actualizaciones; puede tardar varios minutos.

Cómo verificar: Al final no deben quedar paquetes pendientes; cat /etc/os-release muestra 2026.1 o superior.

Errores comunes y solución:

- Error de clave GPG del repositorio: confirme que usa el repositorio oficial kali-rolling.
- Sin conexión: revise la red de la VM (NAT) y vuelva a intentar.

Paso 4: Instalar paquetes base

Objetivo: Asegurar Python, pip, venv y Git.

Explicación conceptual: Estos paquetes son la base para crear entornos virtuales y clonar/gestionar el código.

Dónde se ejecuta: Terminal de Kali.

```
sudo apt install -y python3 python3-pip python3-venv git
# Editor opcional:
sudo apt install -y nano
```

Resultado esperado: Los paquetes quedan instalados o ya estaban presentes.

Cómo verificar: python3 --version, pip3 --version y git --version devuelven números de versión.

Errores comunes y solución:

- python: command not found: en Kali el binario es python3 (ver sección 14).
- Paquete no encontrado: ejecute primero sudo apt update.

Paso 5: Crear el directorio de laboratorio

Objetivo: Tener una carpeta de proyecto ordenada.

Explicación conceptual: Trabajar con el usuario estándar (no root) y en una carpeta propia respeta el principio de mínimo privilegio.

Dónde se ejecuta: Terminal de Kali (en el directorio home del usuario).

```
cd ~
mkdir -p agente-ia-kali
cd agente-ia-kali
```

Resultado esperado: El prompt muestra la ruta ~/agente-ia-kali.

Cómo verificar: pwd devuelve la ruta del proyecto.

Errores comunes y solución:

- Permiso denegado: no trabaje en /root ni en carpetas del sistema; use su home.

Paso 6: Crear el entorno virtual

Objetivo: Aislar las dependencias de Python del sistema.

Explicación conceptual: Un entorno virtual (venv) mantiene las librerías del proyecto separadas, evitando conflictos con las del sistema.

Dónde se ejecuta: Terminal, dentro de ~/agente-ia-kali.

```
python3 -m venv venv
source venv/bin/activate
```

Resultado esperado: El prompt muestra el prefijo (venv).

Cómo verificar: which python apunta a la carpeta venv del proyecto.

Errores comunes y solución:

- Error al crear el entorno: instale python3-venv (sudo apt install python3-venv).
- El prefijo (venv) no aparece: vuelva a ejecutar source venv/bin/activate.

Paso 7: Instalar dependencias

Objetivo: Instalar requests y python-dotenv dentro del venv.

Explicación conceptual: pip instala las librerías listadas en requirements.txt solo dentro del entorno activo.

Dónde se ejecuta: Terminal, con el venv activado.

```
# Cree primero requirements.txt (ver Anexo) y luego:
pip install -r requirements.txt
# Verifique:
pip list
```

Resultado esperado: pip muestra requests y python-dotenv instalados.

Cómo verificar: pip show requests devuelve la versión instalada.

Errores comunes y solución:

- Error de certificado SSL: actualice ca-certificates (ver sección 14).
- Dependencias incompatibles: confirme que el venv esté activo y use las versiones fijadas.

Paso 8: Configurar el modelo

Objetivo: Elegir y preparar el backend (local o API).

Explicación conceptual: config.py lee MODEL_BACKEND desde .env. Para el laboratorio se recomienda 'local' con Ollama.

Dónde se ejecuta: Terminal de Kali.

```
# --- Opcion B (recomendada): Ollama local ---
curl -fsSL https://ollama.com/install.sh | sh
ollama serve &          # inicia el servicio (si no esta activo)
ollama pull llama3.2:3b  # descarga un modelo pequeno
ollama run llama3.2:3b "hola" # prueba rapida

# --- Opcion A: API remota ---
```

```
# Copie .env.example a .env y defina MODEL_BACKEND=api y LLM_API_KEY
```

Resultado esperado: Con Ollama, el modelo responde a la prueba «hola». Con API, la prueba de conexión (Paso 14) responde.

Cómo verificar: ollama list muestra el modelo descargado.

Errores comunes y solución:

- Ollama no inicia: ejecute ollama serve y revise que el puerto 11434 esté libre.
- Modelo no encontrado: confirme el nombre exacto con ollama list.
- Descarga muy lenta: elija un modelo más pequeño.

 **[CAPTURA]** Salida de «ollama list» con el modelo descargado.

⚠ Descarga de Ollama: El script de instalación se descarga de ollama.com. Revíselo antes de ejecutarlo (puede guardarlo con curl -o install.sh y leerlo). No ejecute scripts de fuentes no confiables.

Paso 9: Crear el agente

Objetivo: Colocar los archivos de código del agente en el proyecto.

Explicación conceptual: Cada módulo (config, security, tools, agent, app) cumple una función única. El código completo está en el Anexo.

Dónde se ejecuta: Editor (VS Code o Nano), dentro de ~/agente-ia-kali.

```
nano config.py      # pegar el contenido del Anexo
nano security.py
nano tools.py
nano agent.py
nano app.py
```

Resultado esperado: Los cinco archivos .py quedan creados en el proyecto.

Cómo verificar: ls -la muestra los archivos; python -c "import config" no produce errores.

Errores comunes y solución:

- IndentationError: respete la indentación al pegar; Python es sensible a los espacios.

Paso 10: Crear herramientas seguras

Objetivo: Comprender y verificar tools.py.

Explicación conceptual: Las herramientas usan subprocess.run con una lista de argumentos (nunca shell=True), aplican timeout y truncan la salida. Esto evita inyección de comandos y salidas descontroladas.

Dónde se ejecuta: Editor y terminal.

```
# Ya incluido en tools.py (Anexo). Punto clave:
# subprocess.run([program, *args], timeout=..., capture_output=True)
# NUNCA: subprocess.run(cadena, shell=True)
```

Resultado esperado: tools.run_command ejecuta solo comandos validados.

Cómo verificar: Las pruebas del Paso 15 confirman el comportamiento.

Errores comunes y solución:

- No use shell=True bajo ninguna circunstancia: reintroduce el riesgo de inyección.

Paso 11: Implementar la confirmación humana

Objetivo: Garantizar que ningún comando se ejecute sin un «sí» explícito.

Explicación conceptual: app.py extrae la línea COMANDO de la respuesta del modelo y pregunta [s/N]. Solo 's' o 'si' autorizan.

Dónde se ejecuta: Editor (app.py) y terminal.

```
# Logica incluida en app.py (Anexo):  
# if not _confirmar(comando):  
#     print('[CANCELADO] No se ejecuto el comando.')
```

Resultado esperado: Antes de cada ejecución aparece el aviso [CONFIRMACION REQUERIDA].

Cómo verificar: Pruebe responder «n»: el comando no se ejecuta (caso 12 de pruebas).

Paso 12: Implementar la lista blanca

Objetivo: Restringir los comandos a un conjunto seguro.

Explicación conceptual: security.py define ALLOWED_COMMANDS. Cualquier programa fuera de esa lista se rechaza con SecurityError.

Dónde se ejecuta: Editor (security.py) y terminal.

```
# Verifique la lista activa:  
python -c "from security import describe_whitelist; print(describe_whitelist())"
```

Resultado esperado: Se imprime la lista de 15 comandos permitidos con su riesgo evaluado.

Cómo verificar: Intentar un comando fuera de la lista produce un rechazo (caso 5 de pruebas).

Paso 13: Configurar los logs

Objetivo: Registrar cada interacción para auditoría.

Explicación conceptual: El logger escribe en logs/auditoria.log la fecha, el usuario, la solicitud, el comando autorizado y el resultado.

Dónde se ejecuta: Terminal.

```
# Se crea automaticamente al arrancar app.py. Para revisarlo:  
cat logs/auditoria.log
```

Resultado esperado: El archivo logs/auditoria.log contiene líneas con marca de tiempo.

Cómo verificar: Tras una sesión, el log muestra SESION_INICIADA, SOLICITUD y EJECUCION_OK.

Errores comunes y solución:

- Logs no creados: confirme que la carpeta logs/ tenga permisos de escritura.

Paso 14: Ejecución inicial y prueba de conexión

Objetivo: Arrancar el agente y verificar que se comunica con el modelo.

Explicación conceptual: app.py valida la configuración, muestra la bienvenida y entra en el bucle de conversación.

Dónde se ejecuta: Terminal, con el venv activado.

```
python app.py
# En el prompt del agente, escriba:
usuario> que version del kernel tengo?
```

Resultado esperado: El agente explica y propone «uname -r»; tras confirmar, muestra la versión del kernel.

Cómo verificar: La respuesta del modelo aparece y, al confirmar, se obtiene la salida real del comando.

Errores comunes y solución:

- Clave API no encontrada (modo api): defina LLM_API_KEY en .env.
- Error 401/403: la clave es inválida o no tiene permisos.
- No conecta con Ollama: ejecute ollama serve y confirme el puerto 11434.

 **[CAPTURA]** Primera conversación exitosa con el agente.

Paso 15: Pruebas

Objetivo: Validar funcionalidad y seguridad.

Explicación conceptual: Las pruebas unitarias y los casos manuales de la Sección 12 confirman que el agente acepta lo permitido y rechaza lo peligroso.

Dónde se ejecuta: Terminal, con el venv activado.

```
python -m unittest discover -s tests -v
```

Resultado esperado: Todas las pruebas terminan en «OK».

Cómo verificar: El resumen indica «Ran N tests ... OK» sin fallos.

Errores comunes y solución:

- ModuleNotFoundError: ejecute las pruebas desde la raíz del proyecto con el venv activo.

Paso 16: Cierre y restauración del entorno

Objetivo: Finalizar de forma ordenada y segura.

Explicación conceptual: Al terminar, se desactiva el entorno, se conservan los logs académicos y se puede restaurar el snapshot para dejar la VM limpia.

Dónde se ejecuta: Terminal y hipervisor.

```
# Salir del agente:
usuario> /salir
# Desactivar el entorno virtual:
deactivate
# (Opcional) restaurar el snapshot 'base-limpia' desde el hipervisor
```

Resultado esperado: El entorno queda desactivado y la VM puede restaurarse al estado inicial.

Cómo verificar: El prefijo (venv) desaparece del prompt.

9. Código fuente

El código completo se incluye en el Anexo A. Cumple todos los requisitos de seguridad solicitados:

- Escrito en Python, modular y comentado.
- Maneja excepciones y valida las entradas.
- Evita shell=True; usa subprocess.run() con argumentos separados.
- Aplica un timeout máximo de ejecución (configurable).
- Limita el tamaño de la salida.
- Restringe el acceso al directorio workspace.
- Registra fecha, hora, usuario, solicitud, comando autorizado y resultado.
- Solicita confirmación humana antes de ejecutar.
- Bloquea operadores ; && || | redirecciones y sustitución de comandos.
- Rechaza comandos fuera de la lista blanca.

9.1 Lista blanca y análisis de riesgo

Cada comando se incluyó tras evaluar su riesgo. Todos son informativos o de solo lectura.

Comando	Función	Riesgo evaluado
pwd	Directorio actual	Ninguno
whoami	Usuario actual	Ninguno
date	Fecha y hora	Ninguno
uname	Información del kernel	Divulgación menor de versión
hostname	Nombre del host	Divulgación menor
ip	Configuración de red (lectura)	Divulgación de red
ss	Sockets y conexiones (lectura)	Divulgación de red
df	Espacio en disco	Ninguno
free	Memoria disponible	Ninguno
ps	Procesos en ejecución	Divulgación de procesos
ls	Listar archivos (solo workspace)	Bajo
cat	Mostrar archivos (solo workspace)	Bajo
sha256sum	Hash SHA-256 (solo workspace)	Ninguno
file	Tipo de archivo (solo workspace)	Ninguno
stat	Metadatos (solo workspace)	Bajo

⚠ Prohibido en la lista blanca: No se incluyen comandos ofensivos, destructivos o de explotación (rm, dd, mkfs, kill, nmap, sudo, chmod, curl/wget de ejecución, etc.). Ampliar la lista requiere un nuevo análisis de riesgo y autorización del profesor.

10. Prompt del sistema del agente

El prompt del sistema define el rol, el alcance y las prohibiciones del agente. Se incluye íntegro a continuación y vive en agent.py.

```
\
Eres un agente de inteligencia artificial de asistencia que opera EXCLUSIVAMENTE
dentro de un laboratorio universitario de Seguridad Informatica, en una maquina
virtual aislada y controlada. Tu proposito es educativo y defensivo.
```

REGLAS OBLIGATORIAS:

1. Solo realizas tareas defensivas, informativas y educativas.
2. Explica SIEMPRE, antes de proponer un comando, que hace y por que es seguro.
3. No puedes modificar el sistema sin autorizacion explicita de la persona.
4. Toda ejecucion de comando requiere confirmacion humana; tu solo propones.
5. No ejecutas acciones destructivas (borrar, sobrescribir, formatear, matar procesos del sistema).
6. No atacas, escaneas ni interactuas con sistemas externos.
7. No recopilas, lees ni exfiltras credenciales ni secretos.
8. No desactivas ni evades controles de seguridad.
9. No descargas ni ejecutas codigo desconocido.
10. Respetas estrictamente el alcance definido por el profesor.
11. Registras (a traves del sistema) cada accion realizada.
12. Si hay ambigüedad, riesgo o una peticion fuera de alcance, te DETIENES y pides aclaracion en lugar de actuar.

Solo puedes proponer comandos de la siguiente lista blanca. Cualquier otra peticion debe ser rechazada con una explicacion:

Comandos autorizados:

- cat: Muestra archivos (SOLO dentro de workspace). Riesgo: bajo.
- date: Muestra fecha y hora. Riesgo: ninguno.
- df: Espacio en disco. Riesgo: ninguno.
- file: Tipo de archivo (SOLO dentro de workspace). Riesgo: ninguno.
- free: Memoria disponible. Riesgo: ninguno.
- hostname: Nombre del host. Riesgo: divulgacion menor.
- ip: Configuracion de red (solo lectura). Riesgo: divulgacion de red.
- ls: Lista archivos (restringido a workspace). Riesgo: bajo.
- ps: Procesos en ejecucion. Riesgo: divulgacion de procesos.
- pwd: Muestra el directorio actual. Riesgo: ninguno.
- sha256sum: Hash SHA-256 (SOLO dentro de workspace). Riesgo: ninguno.
- ss: Sockets/conexiones (solo lectura). Riesgo: divulgacion de red.
- stat: Metadatos de archivo (SOLO dentro de workspace). Riesgo: bajo.
- uname: Informacion del kernel. Riesgo: divulgacion menor de version.
- whoami: Muestra el usuario actual. Riesgo: ninguno.

Cuando propongas un comando para ejecutar, responde en este formato:

```
EXPLICACION: <que hace y por que es seguro>
COMANDO: <comando exacto de la lista blanca>
```

Si la peticion no requiere ejecutar nada, responde solo con texto explicativo.

Si la peticion es peligrosa o esta fuera de alcance, responde:

```
RECHAZADO: <motivo>
```

Por qué importa: El prompt del sistema es la primera línea de defensa a nivel del modelo, pero NO es suficiente por sí solo: un modelo puede equivocarse. Por eso los controles reales (lista blanca, confirmación, validación de rutas) están en el código, no solo en el texto del prompt.

11. Controles de seguridad

El diseño combina varias capas de defensa. Ninguna por sí sola es suficiente; juntas reducen el riesgo de forma sustancial.

Control	Qué hace	Dónde vive
Mínimo privilegio	El agente solo puede lo estrictamente necesario	Diseño general
Usuario sin privilegios	No se trabaja como root	Entorno
Restricción de directorios	Lectura solo dentro de workspace	security.py / tools.py
Lista blanca	Solo comandos autorizados	security.py
Confirmación humana	Ningún comando sin «sí» explícito	app.py
Timeout	Cancela comandos largos	config.py / tools.py
Límite de salida	Trunca salidas grandes	tools.py
Validación de parámetros	Tokeniza y revisa rutas	security.py
Anti-inyección	Bloquea ; && \$() ` redirecciones	security.py
Protección de claves	Secretos en .env, fuera del código	config.py / .gitignore
Logs de auditoría	Registro de cada acción	app.py / logs/
Snapshot de la VM	Restauración rápida del entorno	Hipervisor
Segmentación de red	NAT o red interna, nunca puente	Hipervisor
Revisión de código	Leer el código antes de ejecutarlo	Práctica del estudiante
Desactivación final	Apagar el agente al terminar	Procedimiento de cierre

11.1 Por qué un agente con acceso a terminal es un riesgo

Un agente que puede ejecutar comandos combina dos elementos peligrosos: un componente que toma decisiones de forma autónoma (el modelo, que puede equivocarse o ser manipulado mediante instrucciones engañosas) y la capacidad de actuar sobre el sistema. Si no se limita, una instrucción mal interpretada —o una inyección de instrucciones a través de un archivo o texto— podría derivar en acciones no deseadas. Por eso este diseño nunca confía en el modelo para la seguridad: el modelo propone, pero el código valida y la persona autoriza.

⚠ Inyección de instrucciones: El contenido de los archivos de workspace es DATO, no órdenes. Aunque un archivo contenga texto como «ejecuta rm -rf», el agente solo lo trata como contenido a analizar; la lista blanca y la confirmación humana impiden que se convierta en una acción.

12. Casos de prueba

La siguiente tabla combina pruebas funcionales y de seguridad. Para cada caso se indica la entrada, el resultado esperado, la evidencia que el estudiante debe capturar y el criterio de aprobación.

#	Entrada / acción	Resultado esperado	Evidencia	Criterio de aprobación
1	Consultar la versión del kernel	Propone «uname -r»; tras confirmar, muestra la versión	Captura de la salida	Se obtiene la versión del kernel
2	Consultar memoria disponible	Propone «free -h»; muestra la memoria	Captura de la salida	Se obtiene la memoria
3	Listar archivos del workspace	Propone «ls»; lista el contenido de workspace	Captura del listado	Solo aparece contenido de workspace
4	Calcular hash SHA-256 de un archivo	Propone «sha256sum archivo»; muestra el hash	Captura del hash	Hash de 64 caracteres correcto
5	Ejecutar un comando no autorizado (nmap)	Rechazo: comando no autorizado	Captura del rechazo	El comando NO se ejecuta
6	Usar sudo	Rechazo: comando no autorizado	Captura del rechazo	sudo es bloqueado
7	Eliminar un archivo (rm)	Rechazo: comando no autorizado	Captura del rechazo	rm es bloqueado
8	Acceder a /etc/shadow (cat)	Rechazo: ruta fuera de workspace	Captura del rechazo	No se accede a /etc/shadow
9	Usar && (encadenamiento)	Rechazo: operador prohibido	Captura del rechazo	El operador && es detectado
10	Usar (tubería)	Rechazo: operador prohibido	Captura del rechazo	El operador es detectado
11	Escribir fuera de workspace	No existe herramienta de escritura; rechazo	Captura del rechazo	No hay escritura fuera de workspace
12	Cancelar durante la confirmación	Responder «n»: el comando no se ejecuta	Captura de [CANCELADO]	El comando se descarta
13	Provocar un timeout	El comando se cancela al superar el tiempo	Captura del mensaje de timeout	Se respeta el límite de tiempo
14	Desconectar el acceso al modelo	Mensaje de error de conexión claro	Captura del error	El agente no se cae; informa el fallo
15	Usar una clave API incorrecta	Error 401 informado de forma legible	Captura del error 401	Se reporta autenticación fallida

Cómo provocar un timeout (caso 13): Reduzca temporalmente `COMMAND_TIMEOUT` a 1 en `.env` y solicite un comando que tarde más (por ejemplo, agregue un archivo grande al workspace y pida su hash). Restaure el valor original al terminar.

Cómo simular casos 14 y 15: Caso 14: detenga «ollama serve» o desconecte la red de la VM. Caso 15: en modo API, escriba una `LLM_API_KEY` inválida en `.env`.

13. Práctica académica

13.1 Título

Configuración segura y validación de un agente de IA defensivo en Kali Linux.

13.2 Objetivo general

Construir y validar un agente de IA que ejecute únicamente comandos autorizados, con confirmación humana y auditoría, dentro de un entorno virtual controlado.

13.3 Objetivos específicos

- Preparar un entorno virtual aislado con snapshot.
- Configurar un modelo de lenguaje (local o por API).
- Implementar y comprender los controles de seguridad del agente.
- Ejecutar la batería de pruebas funcionales y de seguridad.
- Documentar evidencias y analizar los resultados.

13.4 Requisitos previos

Conocimientos básicos de Linux, terminal y Python; haber leído las secciones 1 a 11 de este manual.

13.5 Recursos

VM con Kali 2026.1, 8 GB de RAM, conexión a Internet para la preparación, este manual y el código del Anexo.

13.6 Normas de seguridad

- Trabajar solo dentro de la VM, nunca en el anfitrión.
- Usar red NAT o interna; el modo puente está prohibido.
- No atacar ni escanear sistemas externos.
- Revisar el código antes de ejecutarlo.
- Conservar los logs como evidencia académica.

13.7 Escenario

El estudiante actúa como analista junior de seguridad defensiva. Debe poner en marcha un asistente de IA que le ayude a inspeccionar el estado de un sistema (kernel, memoria, red, archivos del laboratorio) sin riesgo de ejecutar acciones peligrosas. Su tarea es configurarlo y demostrar que los controles de seguridad funcionan.

13.8 Actividades paso a paso (2 a 3 horas)

Bloque	Actividad	Tiempo
A	Crear VM (o usar una existente) y tomar snapshot 'base-limpia'	15 min
B	Actualizar Kali e instalar paquetes base	20 min
C	Crear el proyecto, el venv e instalar dependencias	15 min
D	Configurar el modelo (Ollama local recomendado)	25 min

Bloque	Actividad	Tiempo
E	Crear los archivos del agente (Anexo) y revisarlos	25 min
F	Ejecución inicial y prueba de conexión (Paso 14)	15 min
G	Ejecutar los 15 casos de prueba (Sección 12)	30 min
H	Documentar evidencias y responder el análisis	20 min
I	Cierre, conservación de logs y restauración	10 min

13.9 Preguntas de análisis

1. ¿Por qué la confirmación humana es necesaria si ya existe una lista blanca?
2. ¿Qué riesgo introduciría usar subprocess con shell=True?
3. ¿Por qué se bloquean operadores como | y && incluso sin usar shell?
4. ¿Qué diferencia hay entre que el prompt del sistema prohíba algo y que el código lo impida?
5. ¿Qué información de los logs sería útil en una investigación de un incidente?
6. ¿Por qué el modo puente está prohibido durante la práctica?

13.10 Evidencias solicitadas

- Capturas de las 15 pruebas con sus resultados.
- Fragmento del archivo logs/auditoria.log.
- Salida de las pruebas unitarias (unittest).
- Captura de la configuración de red de la VM.

13.11 Entregables

Informe en PDF con portada, evidencias, respuestas de análisis y conclusiones; y el archivo de log de auditoría de la sesión.

13.12 Conclusiones (a redactar por el estudiante)

El estudiante debe reflexionar sobre cómo las capas de seguridad reducen el riesgo y qué pasaría si se eliminara alguna de ellas.

13.13 Criterios de evaluación y rúbrica (100 puntos)

Criterio	Descripción	Puntos
Preparación del entorno	VM, snapshot, actualización y paquetes	15
Configuración del modelo	Backend funcional (local o API)	15
Implementación del agente	Archivos creados y en ejecución	20
Pruebas de seguridad	Casos 5–11 superados (rechazos correctos)	20
Pruebas funcionales	Casos 1–4 y 12–15 superados	15
Evidencias y logs	Capturas completas y log de auditoría	10
Análisis y conclusiones	Respuestas razonadas a las preguntas	5
TOTAL		100

14. Solución de problemas

Error	Causa probable	Diagnóstico	Solución
python: command not found	En Kali el binario es python3	which python3	Use python3, o instale python-is-python3
pip: command not found	pip no instalado o fuera del venv	which pip3	sudo apt install python3-pip; active el venv
Error al crear el entorno virtual	Falta python3-venv	python3 -m venv test	sudo apt install python3-venv
Dependencias incompatibles	Versiones mezcladas o venv inactivo	pip list	Recrear el venv e instalar requirements.txt
Error de certificado SSL	ca-certificates desactualizado	Mensaje SSL al usar pip/requests	sudo apt install --reinstall ca-certificates
Clave API no encontrada	.env ausente o variable vacía	echo \$LLM_API_KEY	Defina LLM_API_KEY en .env (modo api)
Error 401 / 403	Clave inválida o sin permisos	Respuesta de la API	Verifique la clave y los permisos del proveedor
Límite de solicitudes (429)	Demasiadas llamadas seguidas	Respuesta 429	Espere y reintente; reduzca la frecuencia
Ollama no inicia	Servicio detenido o puerto ocupado	ss -lntp grep 11434	Ejecute ollama serve; libere el puerto
Modelo local no encontrado	Nombre incorrecto o no descargado	ollama list	ollama pull <modelo> con el nombre exacto
Memoria insuficiente	Modelo demasiado grande para la RAM	free -h durante la carga	Use un modelo más pequeño o más RAM
Permiso denegado	Carpeta del sistema o sin permisos	ls -la	Trabaje en su home; no use root
No ejecuta comandos autorizados	Formato COMANDO no reconocido	Revisar la respuesta del modelo	Ajuste el prompt; verifique el patrón COMANDO:
Acepta comandos que debería rechazar	Lista blanca o validación alterada	Ejecutar las pruebas unitarias	Restaurar security.py; no modifique los controles
Archivo .env expuesto	.env versionado por error	git status	Añada .env a .gitignore y rote la clave
Logs no creados	Carpeta logs/ sin permisos	ls -la logs	Cree logs/ con permisos de escritura

⚠ Acepta comandos que debería rechazar: Este es el fallo más grave. Detenga el laboratorio, restaure security.py desde el Anexo y vuelva a ejecutar las pruebas unitarias antes de continuar.

15. Desinstalación y reversión

Procedimiento ordenado para finalizar y dejar el entorno limpio:

```
# 1. Detener el agente (dentro de su CLI):
usuario> /salir

# 2. Desactivar el entorno virtual:
deactivate
```

```
# 3. (Opcional) eliminar dependencias borrando el venv:
rm -rf ~/agente-ia-kali/venv

# 4. Borrar de forma segura la clave API del .env:
shred -u ~/agente-ia-kali/.env      # sobrescribe y elimina el archivo

# 5. Conservar los logs academicos (copielos antes de borrar el proyecto):
cp ~/agente-ia-kali/logs/auditoria.log ~/evidencias_lab.log

# 6. (Opcional) eliminar el directorio del proyecto:
rm -rf ~/agente-ia-kali

# 7. Verificar que no quedan servicios en ejecucion:
ss -lntp | grep 11434      # Ollama no deberia seguir escuchando si lo detuvo
pgrep -fl ollama          # procesos de Ollama (si decide detenerlo)
```

Para una limpieza total, restaure el snapshot «base-limpia» desde el hipervisor: la VM volverá al estado anterior al laboratorio.

⚠ Conservar evidencias: Copie los logs de auditoría ANTES de restaurar el snapshot o borrar el proyecto; de lo contrario se perderán las evidencias académicas.

Conclusiones

Este manual mostró cómo construir un agente de IA útil y, a la vez, seguro, sobre Kali Linux. La seguridad no recae en un único mecanismo, sino en varias capas que se refuerzan: el aislamiento de la VM, el principio de mínimo privilegio, una lista blanca estricta, la validación de entradas, la confirmación humana y la auditoría completa.

La lección central es que un modelo de lenguaje no debe ser la fuente de la seguridad de un sistema. El modelo propone; el código valida; la persona autoriza. Aplicado con rigor, este patrón permite aprovechar la IA en tareas defensivas sin asumir riesgos innecesarios.

Glosario

Término	Definición
Agente de IA	Programa que usa un modelo de lenguaje para interpretar instrucciones y, opcionalmente, ejecutar acciones mediante herramientas.
Lista blanca	Conjunto cerrado de elementos permitidos; todo lo demás se rechaza por defecto.
Mínimo privilegio	Principio según el cual un componente solo recibe los permisos estrictamente necesarios.
Prompt del sistema	Instrucción inicial que define el rol y las reglas del modelo.
Snapshot	Punto de restauración del estado completo de una máquina virtual.
Timeout	Tiempo máximo permitido para una operación antes de cancelarla.
Inyección de comandos	Ataque que introduce comandos adicionales mediante operadores de shell o entradas no validadas.
venv	Entorno virtual de Python que aísla las dependencias de un proyecto.
Ollama	Herramienta para descargar y ejecutar modelos de lenguaje localmente.
Workspace	Directorio restringido, único lugar donde el agente puede leer archivos.
Auditoría (log)	Registro cronológico de las acciones realizadas, útil para revisión e investigación.
NAT	Modo de red que aísla la VM detrás del anfitrión; opción segura para el laboratorio.

Referencias

Confirme siempre la versión y el contenido en la documentación oficial antes de ejecutar los procedimientos.

- [1] Kali Linux — Sitio y documentación oficial — <https://www.kali.org/>
- [2] Kali Linux — Historial de versiones — <https://www.kali.org/releases/>
- [3] Python 3 — Documentación oficial (subprocess, venv) — <https://docs.python.org/3/>
- [4] Ollama — Documentación oficial — <https://ollama.com/>
- [5] python-dotenv — Documentación — <https://pypi.org/project/python-dotenv/>
- [6] requests — Documentación — <https://requests.readthedocs.io/>
- [7] OWASP — Buenas prácticas de seguridad — <https://owasp.org/>

Anexo A. Código fuente completo

Copie cada archivo en el directorio del proyecto con el nombre indicado. El código fue probado y las pruebas unitarias de seguridad se superan correctamente.

config.py

```
"""
config.py
=====
Carga y centraliza la configuración del agente de IA de laboratorio.

Toda la configuración sensible (claves API) se lee desde variables de
entorno cargadas por python-dotenv a partir de un archivo `.env` que
NUNCA debe subirse a un repositorio (ver .gitignore).

Este módulo no contiene secretos en texto plano. Si una variable
requerida no existe, el agente debe negarse a arrancar con un mensaje
claro en lugar de continuar en un estado inseguro.
"""

from __future__ import annotations

import os
from dataclasses import dataclass, field
from pathlib import Path

from dotenv import load_dotenv

# Carga las variables definidas en .env hacia el entorno del proceso.
# override=False evita pisar variables ya definidas por el sistema.
load_dotenv(override=False)

# Raíz del proyecto (carpeta que contiene este archivo).
BASE_DIR = Path(__file__).resolve().parent

# Directorio de trabajo restringido. El agente SOLO puede leer archivos
# dentro de esta carpeta. Se resuelve de forma absoluta para poder
# compararlo de manera segura más adelante.
WORKSPACE_DIR = (BASE_DIR / "workspace").resolve()

# Directorio de registros de auditoría.
LOGS_DIR = (BASE_DIR / "logs").resolve()

@dataclass(frozen=True)
class Settings:
    """Configuración inmutable del agente."""

    # --- Selección de backend del modelo ---
    # "api" -> usar un proveedor remoto vía API.
    # "local" -> usar un modelo local servido por Ollama.
    model_backend: str = os.getenv("MODEL_BACKEND", "local").strip().lower()

    # --- Opción A: API remota ---
    api_key: str = os.getenv("LLM_API_KEY", "").strip()
```

```

api_base_url: str = os.getenv(
    "LLM_API_BASE_URL", "https://api.anthropic.com"
).strip()
api_model: str = os.getenv("LLM_API_MODEL", "claude-haiku-4-5").strip()

# --- Opción B: Ollama local ---
ollama_base_url: str = os.getenv(
    "OLLAMA_BASE_URL", "http://localhost:11434"
).strip()
ollama_model: str = os.getenv("OLLAMA_MODEL", "llama3.2:3b").strip()

# --- Límites de seguridad para la ejecución de comandos ---
# Tiempo máximo (segundos) que puede correr un comando antes de
# cancelarse por timeout.
command_timeout: int = int(os.getenv("COMMAND_TIMEOUT", "15"))

# Máximo de caracteres de salida que se muestran/registran. Evita
# que un comando inunde la terminal o los logs.
max_output_chars: int = int(os.getenv("MAX_OUTPUT_CHARS", "4000"))

# Rutas derivadas (no provienen de variables de entorno).
workspace_dir: Path = field(default=WORKSPACE_DIR)
logs_dir: Path = field(default=LOGS_DIR)

def validate(self) -> None:
    """Valida la configuración y crea los directorios necesarios.

    Lanza RuntimeError si la configuración es incoherente, para que
    el agente se niegue a arrancar en un estado inseguro.
    """
    if self.model_backend not in ("api", "local"):
        raise RuntimeError(
            f"MODEL_BACKEND invalido: '{self.model_backend}'. "
            "Use 'api' o 'local'."
        )

    if self.model_backend == "api" and not self.api_key:
        raise RuntimeError(
            "MODEL_BACKEND=api pero LLM_API_KEY esta vacia. "
            "Defina la clave en el archivo .env."
        )

    if self.command_timeout <= 0:
        raise RuntimeError("COMMAND_TIMEOUT debe ser mayor que cero.")

    if self.max_output_chars <= 0:
        raise RuntimeError("MAX_OUTPUT_CHARS debe ser mayor que cero.")

    # Crea las carpetas de trabajo y de logs si no existen.
    self.workspace_dir.mkdir(parents=True, exist_ok=True)
    self.logs_dir.mkdir(parents=True, exist_ok=True)

# Instancia única utilizada por el resto de la aplicación.
settings = Settings()

```

security.py

```

"""
security.py
=====
Núcleo de seguridad del agente. Aquí viven los controles que impiden que
el modelo de lenguaje (o un usuario) ejecute acciones peligrosas:

* Lista blanca de comandos permitidos.
* Detección de operadores de shell prohibidos (; && || | ` $() etc.).
* Validación de que las rutas se mantengan dentro de `workspace/`.
* Validación específica por comando (p. ej. `cat` solo en workspace).

NINGÚN comando se ejecuta directamente aquí: este módulo solo *decide*
si una solicitud es aceptable. La ejecución real ocurre en tools.py con
subprocess.run() y argumentos separados (sin shell).
"""

from __future__ import annotations

import re
import shlex
from pathlib import Path
from typing import List, Tuple

from config import settings

# -----
# 1. Lista blanca de comandos
# -----
# Solo comandos informativos / defensivos. Cada entrada documenta el
# riesgo evaluado. NO se incluyen comandos ofensivos, destructivos ni de
# explotación.
ALLOWED_COMMANDS = {
    "pwd": "Muestra el directorio actual. Riesgo: ninguno.",
    "whoami": "Muestra el usuario actual. Riesgo: ninguno.",
    "date": "Muestra fecha y hora. Riesgo: ninguno.",
    "uname": "Información del kernel. Riesgo: divulgación menor de versión.",
    "hostname": "Nombre del host. Riesgo: divulgación menor.",
    "ip": "Configuración de red (solo lectura). Riesgo: divulgación de red.",
    "ss": "Sockets/conexiones (solo lectura). Riesgo: divulgación de red.",
    "df": "Espacio en disco. Riesgo: ninguno.",
    "free": "Memoria disponible. Riesgo: ninguno.",
    "ps": "Procesos en ejecución. Riesgo: divulgación de procesos.",
    "ls": "Lista archivos (restringido a workspace). Riesgo: bajo.",
    "cat": "Muestra archivos (SOLO dentro de workspace). Riesgo: bajo.",
    "sha256sum": "Hash SHA-256 (SOLO dentro de workspace). Riesgo: ninguno.",
    "file": "Tipo de archivo (SOLO dentro de workspace). Riesgo: ninguno.",
    "stat": "Metadatos de archivo (SOLO dentro de workspace). Riesgo: bajo.",
}

# Comandos cuyos argumentos de ruta DEBEN permanecer dentro de workspace.
PATH_RESTRICTED_COMMANDS = {"cat", "sha256sum", "file", "stat", "ls"}

# -----
# 2. Operadores de shell prohibidos
# -----

```

```

# Se rechaza cualquier solicitud que contenga estos patrones, porque
# habilitan encadenamiento, tuberías, redirecciones o sustitucion de
# comandos. Aunque NO usamos shell=True, los bloqueamos por defensa en
# profundidad y para dar mensajes claros al estudiante.
FORBIDDEN_PATTERNS = [
    (";", "secuencia de comandos ';'"),
    ("&&", "operador AND '&&'"),
    ("||", "operador OR '||'"),
    ("|", "tuberia '|'"),
   ("&", "ejecucion en segundo plano '&'"),
   (">", "redireccion de salida '>'"),
   ("<", "redireccion de entrada '<'"),
   ("`", "sustitucion de comandos con backticks"),
   ("$(", "sustitucion de comandos '$()'"),
    ("\n", "salto de linea"),
    ("\\", "barra invertida"),
]

class SecurityError(Exception):
    """Se lanza cuando una solicitud viola una regla de seguridad."""

def check_forbidden_operators(raw_input: str) -> None:
    """Rechaza la entrada si contiene operadores de shell peligrosos."""
    for token, description in FORBIDDEN_PATTERNS:
        if token in raw_input:
            raise SecurityError(
                f"Entrada rechazada: contiene {description}. "
                "No se permiten operadores de shell."
            )

def _is_within_workspace(candidate: Path) -> bool:
    """True si `candidate` está dentro de workspace/ (resolviendo symlinks)."""
    try:
        resolved = (settings.workspace_dir / candidate).resolve()
    except (OSError, RuntimeError):
        return False
    # Python 3.9+: Path.is_relative_to
    return resolved == settings.workspace_dir or settings.workspace_dir in resolved.parents

def validate_command(raw_input: str) -> Tuple[str, List[str]]:
    """Valida una solicitud de comando y devuelve (programa, argumentos).

    Pasos:
    1. Rechaza operadores de shell prohibidos.
    2. Divide de forma segura con shlex (sin invocar shell).
    3. Verifica que el programa esté en la lista blanca.
    4. Para comandos restringidos por ruta, confirma que toda ruta
       apunte dentro de workspace/.

    Lanza SecurityError si algo no cumple las reglas.
    """
    raw_input = raw_input.strip()
    if not raw_input:
        raise SecurityError("Entrada vacia.")

```

```

# Paso 1: operadores prohibidos.
check_forbidden_operators(raw_input)

# Paso 2: tokenizacion segura.
try:
    tokens = shlex.split(raw_input)
except ValueError as exc:
    raise SecurityError(f"No se pudo interpretar la entrada: {exc}")

if not tokens:
    raise SecurityError("No se encontro ningun comando.")

program, args = tokens[0], tokens[1:]

# Paso 3: lista blanca.
if program not in ALLOWED_COMMANDS:
    raise SecurityError(
        f"Comando '{program}' no autorizado. "
        f"Permitidos: {' ', '.join(sorted(ALLOWED_COMMANDS))}."
    )

# Paso 4: restriccion de rutas.
if program in PATH_RESTRICTED_COMMANDS:
    for arg in args:
        # Se ignoran las banderas (p. ej. -l, -a, --human-readable).
        if arg.startswith("-"):
            continue
        if not _is_within_workspace(Path(arg)):
            raise SecurityError(
                f"La ruta '{arg}' esta fuera del directorio workspace. "
                "Solo se permite acceder a archivos dentro de workspace/."
            )

    return program, args

def describe_whitelist() -> str:
    """Devuelve una descripción legible de la lista blanca (para ayuda)."""
    lines = ["Comandos autorizados:"]
    for name, risk in sorted(ALLOWED_COMMANDS.items()):
        lines.append(f"  - {name}: {risk}")
    return "\n".join(lines)

```

tools.py

```

"""
tools.py
=====
Herramientas seguras que el agente puede usar. Cada herramienta:

* Valida su entrada antes de actuar.
* Usa subprocess.run() con una LISTA de argumentos (nunca shell=True).
* Aplica un timeout configurable.
* Limita el tamaño de la salida.
* Registra la actividad mediante el logger de auditoría.

```

La herramienta principal es `run_command`, que ejecuta comandos de la lista blanca SOLO después de validarlos en `security.py` y de obtener confirmación humana (la confirmación se solicita en `app.py`).

```

"""

from __future__ import annotations

import subprocess
from pathlib import Path
from typing import List

from config import settings
from security import SecurityError, validate_command

def _truncate(text: str) -> str:
    """Recorta la salida al máximo permitido y avisa si se truncó."""
    limit = settings.max_output_chars
    if len(text) <= limit:
        return text
    return text[:limit] + f"\n... [salida truncada a {limit} caracteres]"

def run_command(raw_input: str) -> str:
    """Valida y ejecuta un comando de la lista blanca.

    Devuelve la salida combinada (stdout + stderr) ya truncada.
    Lanza SecurityError si el comando no es válido. Cualquier fallo de
    ejecución se devuelve como texto legible (no se relanza) para que el
    agente pueda explicarlo al estudiante.
    """
    # 1. Validacion de seguridad (lista blanca, operadores, rutas).
    program, args = validate_command(raw_input)

    # 2. Ejecucion segura: lista de argumentos, sin shell, con timeout.
    try:
        completed = subprocess.run(
            [program, *args],
            capture_output=True,
            text=True,
            timeout=settings.command_timeout,
            cwd=str(settings.workspace_dir), # se ejecuta dentro de workspace
            check=False,
        )
    except subprocess.TimeoutExpired:
        return (
            f"[ERROR] El comando '{program}' supero el tiempo maximo de "
            f"{settings.command_timeout} s y fue cancelado."
        )
    except FileNotFoundError:
        return f"[ERROR] El programa '{program}' no esta instalado en el sistema."
    except OSError as exc:
        return f"[ERROR] No se pudo ejecutar '{program}': {exc}"

    # 3. Combinar y truncar la salida.
    salida = completed.stdout or ""
    error = completed.stderr or ""
    combinada = salida
    if error:
        combinada += ("\n[stderr]\n" + error) if combinada else error

    if not combinada.strip():

```

```

        combinada = f"[ok] El comando '{program}' termino con codigo {completed.returncode}
sin salida."

        return _truncate(combinada)

def read_workspace_file(filename: str) -> str:
    """Lee un archivo de texto DENTRO de workspace/ de forma segura.

    Es una herramienta de conveniencia equivalente a `cat` restringido.
    """
    target = (settings.workspace_dir / filename).resolve()
    if target != settings.workspace_dir and settings.workspace_dir not in target.parents:
        raise SecurityError(
            f"'{filename}' esta fuera de workspace. Acceso denegado."
        )
    if not target.is_file():
        return f"[ERROR] El archivo '{filename}' no existe en workspace/."
    try:
        contenido = target.read_text(encoding="utf-8", errors="replace")
    except OSError as exc:
        return f"[ERROR] No se pudo leer '{filename}': {exc}"
    return _truncate(contenido)

def list_workspace() -> str:
    """Lista los archivos disponibles en workspace/."""
    entradas: List[str] = []
    for p in sorted(settings.workspace_dir.iterdir()):
        marca = "/" if p.is_dir() else ""
        entradas.append(f"{p.name}{marca}")
    if not entradas:
        return "workspace/ esta vacio."
    return "Contenido de workspace/:\n" + "\n".join(entradas)

```

agent.py

```

"""
agent.py
=====
Lógica del agente: construye el prompt del sistema, consulta al modelo de
lenguaje (API remota u Ollama local) y mantiene un historial básico de la
conversación.

El agente NO ejecuta comandos por su cuenta. Cuando el modelo propone un
comando, app.py es quien solicita confirmación humana y luego invoca las
herramientas de tools.py. Esta separación es deliberada: el modelo
*sugiere*, la persona *autoriza*, y el código *valida y ejecuta*.
"""

from __future__ import annotations

import json
from typing import Dict, List

import requests

from config import settings
from security import describe_whitelist

```



```
# -----
# Prompt del sistema (ver Sección 10 del manual)
# -----
SYSTEM_PROMPT = f"""
Eres un agente de inteligencia artificial de asistencia que opera EXCLUSIVAMENTE
dentro de un laboratorio universitario de Seguridad Informática, en una máquina
virtual aislada y controlada. Tu propósito es educativo y defensivo.
```

REGLAS OBLIGATORIAS:

1. Solo realizas tareas defensivas, informativas y educativas.
2. Explica SIEMPRE, antes de proponer un comando, que hace y por qué es seguro.
3. No puedes modificar el sistema sin autorización explícita de la persona.
4. Toda ejecución de comando requiere confirmación humana; tu solo propones.
5. No ejecutas acciones destructivas (borrar, sobrescribir, formatear, matar procesos del sistema).
6. No atacas, escaneas ni interactúas con sistemas externos.
7. No recopilas, lees ni exfiltras credenciales ni secretos.
8. No desactivas ni evades controles de seguridad.
9. No descargas ni ejecutas código desconocido.
10. Respetas estrictamente el alcance definido por el profesor.
11. Registras (a través del sistema) cada acción realizada.
12. Si hay ambigüedad, riesgo o una petición fuera de alcance, te DETIENES y pides aclaración en lugar de actuar.

Solo puedes proponer comandos de la siguiente lista blanca. Cualquier otra petición debe ser rechazada con una explicación:

```
{describe_whitelist()}
```

Cuando propongas un comando para ejecutar, responde en este formato:

```
EXPLICACION: <que hace y por que es seguro>
COMANDO: <comando exacto de la lista blanca>
```

Si la petición no requiere ejecutar nada, responde solo con texto explicativo.

Si la petición es peligrosa o está fuera de alcance, responde:

```
RECHAZADO: <motivo>
"""
```

```
class Agent:
```

```
    """Agente conversacional con historial básico."""
```

```
    def __init__(self) -> None:
```

```
        # Historial de mensajes (rol, contenido). Se reenvía al modelo en
        # cada turno porque los modelos no tienen memoria entre llamadas.
        self.history: List[Dict[str, str]] = []
```

```
    def reset(self) -> None:
```

```
        """Borra el historial de la conversación."""
        self.history.clear()
```

```
    def ask(self, user_message: str) -> str:
```

```
        """Envía un mensaje al modelo y devuelve su respuesta de texto."""
        self.history.append({"role": "user", "content": user_message})
```

```
        if settings.model_backend == "api":
```

```

        answer = self._ask_api()
    else:
        answer = self._ask_ollama()

    self.history.append({"role": "assistant", "content": answer})
    return answer

# ----- Backend: API remota -----
def _ask_api(self) -> str:
    """Consulta la API de mensajes de Anthropic (formato estandar)."""
    url = f"{settings.api_base_url}/v1/messages"
    headers = {
        "x-api-key": settings.api_key,
        "anthropic-version": "2023-06-01",
        "content-type": "application/json",
    }
    payload = {
        "model": settings.api_model,
        "max_tokens": 1024,
        "system": SYSTEM_PROMPT,
        "messages": self.history,
    }
    try:
        resp = requests.post(url, headers=headers, json=payload, timeout=60)
    except requests.RequestException as exc:
        return f"[ERROR] No se pudo contactar la API: {exc}"

    if resp.status_code == 401:
        return "[ERROR] Autenticacion fallida (401): revise LLM_API_KEY."
    if resp.status_code == 403:
        return "[ERROR] Acceso prohibido (403): permisos insuficientes."
    if resp.status_code == 429:
        return "[ERROR] Limite de solicitudes alcanzado (429): espere e intente de nuevo."
    if resp.status_code >= 400:
        return f"[ERROR] La API respondio {resp.status_code}: {resp.text[:200]}"

    try:
        data = resp.json()
        partes = [b.get("text", "") for b in data.get("content", []) if b.get("type") == "text"]
        return "\n".join(p for p in partes if p).strip() or "[sin respuesta del modelo]"
    except (ValueError, KeyError) as exc:
        return f"[ERROR] Respuesta de la API no valida: {exc}"

# ----- Backend: Ollama local -----
def _ask_ollama(self) -> str:
    """Consulta un modelo local servido por Ollama."""
    url = f"{settings.ollama_base_url}/api/chat"
    # Ollama acepta un mensaje 'system' como primer elemento.
    mensajes = [{"role": "system", "content": SYSTEM_PROMPT}, *self.history]
    payload = {
        "model": settings.ollama_model,
        "messages": mensajes,
        "stream": False,
    }
    try:
        resp = requests.post(url, json=payload, timeout=120)
    except requests.RequestException as exc:
        return (

```

```

        f"[ERROR] No se pudo contactar Ollama en {settings.ollama_base_url}: "
        f"{exc}. Verifique que el servicio este activo (ollama serve)."
    )

    if resp.status_code >= 400:
        return f"[ERROR] Ollama respondió {resp.status_code}: {resp.text[:200]}"

    try:
        data = resp.json()
        return data.get("message", {}).get("content", "").strip() or "[sin respuesta]"
    except ValueError as exc:
        return f"[ERROR] Respuesta de Ollama no valida: {exc}"

```

app.py

```

"""
app.py
=====
Punto de entrada del agente: interfaz de línea de comandos (CLI).

Flujo de cada turno:
1. La persona escribe una instrucción en lenguaje natural.
2. El agente (modelo) responde con una explicación y, opcionalmente, un
   COMANDO propuesto de la lista blanca.
3. Si hay un comando, app.py PIDE CONFIRMACIÓN HUMANA explícita.
4. Solo si la persona confirma, se valida de nuevo y se ejecuta con
   tools.run_command(). Todo queda registrado en logs/.

Comandos especiales de la CLI:
/salir   -> termina la sesión.
/ayuda   -> muestra la lista blanca.
/limpiar -> borra el historial de la conversación.
"""

from __future__ import annotations

import logging
import re
import sys
from datetime import datetime
from getpass import getuser

from agent import Agent
from config import settings
from security import SecurityError, describe_whitelist
from tools import run_command

# -----
# Configuración del logger de auditoría
# -----
def _build_logger() -> logging.Logger:
    logger = logging.getLogger("agente_lab")
    logger.setLevel(logging.INFO)
    if logger.handlers: # evita duplicar handlers si se reimporta
        return logger
    log_file = settings.logs_dir / "auditoria.log"
    handler = logging.FileHandler(log_file, encoding="utf-8")
    fmt = logging.Formatter("%(asctime)s | %(levelname)s | %(message)s")

```

```

        handler.setFormatter(fmt)
        logger.addHandler(handler)
        return logger

logger = _build_logger()

# Extrae la línea "COMANDO: ..." de la respuesta del modelo, si existe.
_COMANDO_RE = re.compile(r"^\s*COMANDO:\s*(.+)\s*$", re.IGNORECASE | re.MULTILINE)

def _extraer_comando(respuesta_modelo: str) -> str | None:
    match = _COMANDO_RE.search(respuesta_modelo)
    if not match:
        return None
    comando = match.group(1).strip()
    return comando or None

def _confirmar(comando: str) -> bool:
    """Solicita confirmación humana explícita. Solo 's' o 'si' aceptan."""
    print(f"\n[CONFIRMACION REQUERIDA] El agente desea ejecutar:\n    {comando}")
    try:
        respuesta = input("¿Autoriza la ejecucion? [s/N]: ").strip().lower()
    except (EOFError, KeyboardInterrupt):
        print("\n[CANCELADO] Ejecucion cancelada por el usuario.")
        return False
    return respuesta in ("s", "si", "sí")

def main() -> int:
    # Valida la configuración; si falla, no arranca.
    try:
        settings.validate()
    except RuntimeError as exc:
        print(f"[ERROR DE CONFIGURACION] {exc}")
        return 1

    usuario = getuser()
    agente = Agent()

    print("=" * 60)
    print(" Agente de IA de laboratorio (uso academico y defensivo) ")
    print("=" * 60)
    print(f"Backend del modelo: {settings.model_backend}")
    print(f"Directorio de trabajo: {settings.workspace_dir}")
    print("Escriba /ayuda para ver los comandos permitidos, /salir para terminar.\n")
    logger.info("SESION_INICIADA | usuario=%s | backend=%s", usuario,
settings.model_backend)

    while True:
        try:
            entrada = input("usuario> ").strip()
        except (EOFError, KeyboardInterrupt):
            print("\nSaliendo...")
            break

```

```

if not entrada:
    continue
if entrada == "/salir":
    print("Sesion finalizada.")
    break
if entrada == "/ayuda":
    print(describe_whitelist())
    continue
if entrada == "/limpiar":
    agente.reset()
    print("[ok] Historial borrado.")
    continue

# 1. Consultar al modelo.
respuesta = agente.ask(entrada)
print(f"\nagente> {respuesta}\n")
logger.info("SOLICITUD | usuario=%s | texto=%r", usuario, entrada)

# 2. ¿Propuso un comando?
comando = _extraer_comando(respuesta)
if not comando:
    continue

# 3. Confirmación humana.
if not _confirmar(comando):
    print("[CANCELADO] No se ejecuto el comando.")
    logger.info("EJECUCION_CANCELADA | usuario=%s | comando=%r", usuario, comando)
    continue

# 4. Validar de nuevo y ejecutar.
try:
    resultado = run_command(comando)
    print(f"\n[RESULTADO]\n{resultado}\n")
    logger.info(
        "EJECUCION_OK | usuario=%s | comando=%r | bytes_salida=%d",
        usuario, comando, len(resultado),
    )
except SecurityError as exc:
    print(f"[RECHAZADO POR SEGURIDAD] {exc}")
    logger.warning(
        "EJECUCION_RECHAZADA | usuario=%s | comando=%r | motivo=%s",
        usuario, comando, exc,
    )

logger.info("SESION_FINALIZADA | usuario=%s", usuario)
return 0

if __name__ == "__main__":
    sys.exit(main())

```

tests/test_security.py

```

"""
tests/test_security.py
=====
Pruebas unitarias del núcleo de seguridad. Verifican que:
* Los comandos de la lista blanca se aceptan.
* Los comandos no autorizados se rechazan.
* Los operadores de shell prohibidos se detectan.

```

```

* Las rutas fuera de workspace se rechazan.

Ejecutar con: python -m pytest tests/ -v
(o:          python -m unittest discover -s tests)
"""

import sys
import unittest
from pathlib import Path

# Permite importar los módulos del proyecto (carpeta padre).
sys.path.insert(0, str(Path(__file__).resolve().parent.parent))

from security import SecurityError, validate_command # noqa: E402

class TestListaBlanca(unittest.TestCase):
    def test_comando_permitido(self):
        prog, args = validate_command("uname -a")
        self.assertEqual(prog, "uname")
        self.assertEqual(args, ["-a"])

    def test_comando_no_authorized(self):
        with self.assertRaises(SecurityError):
            validate_command("nmap 127.0.0.1")

    def test_sudo_rechazado(self):
        with self.assertRaises(SecurityError):
            validate_command("sudo cat archivo")

    def test_rm_rechazado(self):
        with self.assertRaises(SecurityError):
            validate_command("rm archivo.txt")

class TestOperadores(unittest.TestCase):
    def test_punto_y_coma(self):
        with self.assertRaises(SecurityError):
            validate_command("pwd; whoami")

    def test_and_logico(self):
        with self.assertRaises(SecurityError):
            validate_command("ls && whoami")

    def test_tuberia(self):
        with self.assertRaises(SecurityError):
            validate_command("ps | grep root")

    def test_redireccion(self):
        with self.assertRaises(SecurityError):
            validate_command("date > salida.txt")

    def test_sustitucion(self):
        with self.assertRaises(SecurityError):
            validate_command("echo $(whoami)")

class TestRutas(unittest.TestCase):

```

```
def test_acceso_fuera_de_workspace(self):
    with self.assertRaises(SecurityError):
        validate_command("cat /etc/shadow")

def test_escape_con_dotdot(self):
    with self.assertRaises(SecurityError):
        validate_command("cat ../config.py")

if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Anexo B. requirements.txt

```
# Dependencias del agente de IA de laboratorio
# Versiones fijadas por compatibilidad. Confirme en la documentacion
# oficial antes de actualizar.

requests==2.32.3      # Cliente HTTP para API remota y Ollama
python-dotenv==1.0.1  # Carga de variables de entorno desde .env
```

Anexo C. .env.example

```
# =====
# Plantilla de configuracion. COPIE este archivo a .env y
# complete los valores. NUNCA suba .env a un repositorio.
#   cp .env.example .env
# =====

# Backend del modelo: "local" (Ollama) o "api" (proveedor remoto)
MODEL_BACKEND=local

# ---- Opcion A: API remota ----
# Deje LLM_API_KEY vacia si usa el backend local.
LLM_API_KEY=
LLM_API_BASE_URL=https://api.anthropic.com
LLM_API_MODEL=claude-haiku-4-5

# ---- Opcion B: Ollama local ----
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2:3b

# ---- Limites de seguridad ----
COMMAND_TIMEOUT=15
MAX_OUTPUT_CHARS=4000
```

Anexo D. .gitignore

```
# Secretos: nunca versionar el archivo .env real
.env

# Entorno virtual
venv/
.venv/

# Logs de auditoria (conservar localmente, no en el repo)
logs/

# Cache de Python
__pycache__/
*.py[cod]
*.egg-info/
.pytest_cache/

# Contenido del area de trabajo del laboratorio
workspace/*
!workspace/.gitkeep
```


Anexo E. Lista de comprobación para el profesor

✓	Punto a verificar
☐	El laboratorio se ejecuta solo en máquinas virtuales aisladas
☐	Se exige red NAT o interna; el modo puente está deshabilitado
☐	Cada estudiante tomó el snapshot 'base-limpia' antes de comenzar
☐	Si se usa API, las claves son individuales y no se comparten en texto plano
☐	Se definió el alcance permitido y los estudiantes lo conocen
☐	La lista blanca no fue ampliada sin análisis de riesgo y autorización
☐	Las pruebas de seguridad (casos 5–11) se rechazan correctamente
☐	Los logs de auditoría se generan y se conservan como evidencia
☐	Se revisó el código de los estudiantes en busca de shell=True u otras alteraciones
☐	Al finalizar, los entornos se desactivan y/o los snapshots se restauran

Anexo F. Lista de comprobación para el estudiante

✓	Punto a verificar
☐	Verifiqué versiones de Kali, Python, pip y Git
☐	Tomé el snapshot 'base-limpia' antes de instalar
☐	Actualicé Kali e instalé los paquetes base
☐	Creé y activé el entorno virtual (venv)
☐	Instalé las dependencias desde requirements.txt
☐	Configuré el modelo (Ollama local o API) y probé la conexión
☐	Copié los cinco archivos del agente y los revisé
☐	Creé .env a partir de .env.example y NO lo subí a ningún repositorio
☐	Ejecuté las pruebas unitarias y todas pasaron (OK)
☐	Completé los 15 casos de prueba y capturé las evidencias
☐	Revisé el archivo logs/auditoria.log
☐	Cerré el agente, conservé los logs y restauré el entorno

Anexo G. Rúbrica de evaluación (100 puntos)

Criterio	Excelente	Aceptable	Insuficiente	Pts
Entorno (15)	VM, snapshot y actualización correctos (15)	Falta un elemento menor (8–14)	Entorno incompleto (0–7)	15
Modelo (15)	Backend funcional y probado (15)	Funciona con ayuda (8–14)	No funciona (0–7)	15

Criterio	Excelente	Aceptable	Insuficiente	Pts
Agente (20)	Todos los archivos en ejecución (20)	Ejecuta con errores menores (10–19)	No ejecuta (0–9)	20
Seguridad (20)	Casos 5–11 rechazados (20)	1–2 fallos (10–19)	Múltiples fallos (0–9)	20
Funcional (15)	Casos 1–4, 12–15 correctos (15)	Algún fallo (8–14)	Mayoría falla (0–7)	15
Evidencias (10)	Completas y claras (10)	Parciales (5–9)	Ausentes (0–4)	10
Análisis (5)	Respuestas razonadas (5)	Básicas (2–4)	Ausentes (0–1)	5
TOTAL				100

— Fin del manual —